2026–2027 PROFESSIONAL EDITION (VERSION 2.0)

CKA Mastery Blueprint: Premium Features Suite

Readiness Assessment, Study Plans, Flashcards, Tracker, Mistake Journal & Interview Guide

Cloud Native Engineering Press

Targeting Kubernetes v1.31.x / v1.32.x (Forward Compatible through v1.33+)

Independent Professional Certification Study System

Certified Kubernetes Administrator (CNCF / The Linux Foundation)

CKA READINESS ASSESSMENT & DIAGNOSTIC SYSTEM

INITIAL KNOWLEDGE EVALUATION & PERSONALIZED STUDY ROADMAP

---

1. ASSESSMENT OBJECTIVE & METHODOLOGY

Before embarking on your CKA preparation journey, taking this 25-question diagnostic assessment establishes your baseline across the 5 official CKA domains.

- **Format:** Multiple-choice scenario questions with single best answer.
- **Scoring:** 4 points per correct answer (Total: 100 points).
- **Passing Benchmark:** 70 points indicates intermediate readiness; below 50 points recommends starting with the 60-Day Beginner Track.

---

2. DIAGNOSTIC ASSESSMENT QUESTIONS

DOMAIN 1: CLUSTER ARCHITECTURE, INSTALLATION & CONFIGURATION (25%)

Q1. When executing `kubeadm init`, which flag is mandatory to allow other control plane nodes to securely join an HA cluster?

- [ ] A. `--experimental-upload-certs`
- [ ] B. `--upload-certs`
- [ ] C. `--ha-enable`
- [ ] D. `--control-plane-sync`

Q2. An etcd cluster consists of 5 members. How many simultaneous member failures can this cluster tolerate while maintaining full write availability?

- [ ] A. 1
- [ ] B. 2
- [ ] C. 3
- [ ] D. 4

Q3. You need to safely upgrade a worker node from v1.31.0 to v1.31.2. Which command must be executed on the control plane *before* upgrading packages on the worker node?

- [ ] A. `kubectl cordon <node>`
- [ ] B. `kubectl drain <node> --ignore-daemonsets --delete-emptydir-data`
- [ ] C. `kubectl delete node <node>`
- [ ] D. `kubeadm upgrade node`

Q4. A ServiceAccount named `deployer` in namespace `qa` needs to create deployments in namespace `staging`. Which RBAC resource must be created?

- [ ] A. `Role` in `qa` and `RoleBinding` in `qa`.
- [ ] B. `ClusterRole` and `ClusterRoleBinding`.
- [ ] C. `Role` in `staging` and `RoleBinding` in `staging` referencing `qa:deployer` in subjects.
- [ ] D. `Role` in `staging` and `RoleBinding` in `qa`.

Q5. Which command is used to renew control plane certificates created by kubeadm?

- [ ] A. `kubeadm alpha certs renew`
- [ ] B. `kubeadm certs renew all`
- [ ] C. `kubectl cert-manager renew`
- [ ] D. `openssl req -renew`

DOMAIN 2: WORKLOADS & SCHEDULING (15%)

Q6. A Deployment rollout of `app-v2` is failing due to CrashLoopBackOff. Which command reverts the deployment to revision 1 without editing YAML?

- [ ] A. `kubectl rollout undo deployment app --to-revision=1`
- [ ] B. `kubectl rollback deployment app -v 1`
- [ ] C. `kubectl set image deployment app app=app:v1`
- [ ] D. `kubectl revert deployment app 1`

Q7. A container in a Pod is terminated with exit code `137`. What is the primary cause?

- [ ] A. Application segmentation fault.
- [ ] B. Memory consumption exceeded `resources.limits.memory` (OOMKilled).
- [ ] C. Liveness probe failed 3 consecutive times.
- [ ] D. Missing ConfigMap environment variable.

Q8. You want to ensure that Pods of Deployment `api` are never co-located on the same physical host as another `api` Pod. Which feature must be declared?

- [ ] A. `nodeSelector`
- [ ] B. `podAntiAffinity` with `topologyKey: kubernetes.io/hostname`
- [ ] C. `topologySpreadConstraints` with `maxSkew: 0`
- [ ] D. `nodeAffinity` with `NotIn` operator

Q9. A batch Job has `completions: 5` and `parallelism: 2`. How many Pods run simultaneously?

- [ ] A. 1
- [ ] B. 2
- [ ] C. 5
- [ ] D. 10

DOMAIN 3: SERVICES & NETWORKING (20%)

Q10. What is the fully qualified domain name (FQDN) for a Service named `auth-svc` located in namespace `security` using default cluster domain?

- [ ] A. `auth-svc.security.cluster.local`
- [ ] B. `auth-svc.security.svc.cluster.local`
- [ ] C. `auth-svc.svc.security.cluster`
- [ ] D. `security.auth-svc.k8s.local`

Q11. When using the Gateway API, which resource decouples infrastructure listeners (ports and TLS) from developer routing paths?

- [ ] A. `Ingress`
- [ ] B. `GatewayClass`
- [ ] C. `Gateway`
- [ ] D. `HTTPRoute`

Q12. An HTTPRoute in namespace `frontend` needs to bind to a Gateway in namespace `infra`. Which resource must exist in namespace `infra` to permit this cross-namespace attachment?

- [ ] A. `NetworkPolicy`
- [ ] B. `ReferenceGrant`
- [ ] C. `ClusterRoleBinding`
- [ ] D. `SecretExport`

Q13. A namespace has a NetworkPolicy with `podSelector: {}` and `policyTypes: [Ingress]`, but no `ingress:` rules defined. What is the effect?

- [ ] A. All ingress traffic is allowed.
- [ ] B. All ingress traffic to all pods in the namespace is denied (Default Deny).
- [ ] C. Traffic from the same namespace is allowed, external traffic is denied.
- [ ] D. DNS traffic is blocked.

Q14. In a NetworkPolicy `ingress.from` clause, what is the semantic difference between listing `namespaceSelector` and `podSelector` under a single list item versus two separate hyphens?

- [ ] A. Single item is OR; separate hyphens is AND.
- [ ] B. Single item is AND; separate hyphens is OR.
- [ ] C. Both syntaxes produce identical AND behavior.

- [ ] D. Separate hyphens are invalid syntax.

DOMAIN 4: STORAGE (10%)

Q15. A StorageClass has `volumeBindingMode: WaitForFirstConsumer`. When is the PersistentVolume dynamically provisioned?

- [ ] A. Immediately when the PersistentVolumeClaim is created.
- [ ] B. When a Pod consuming the PVC is scheduled to a specific node.
- [ ] C. When the first container inside the Pod writes data.
- [ ] D. When the node reboots.

Q16. Which volume access mode guarantees that only a single Pod in the entire cluster can mount the volume read-write?

- [ ] A. `ReadWriteOnce` (RWO)
- [ ] B. `ReadOnlyMany` (ROX)
- [ ] C. `ReadWriteMany` (RWX)
- [ ] D. `ReadWriteOncePod` (RWOP)

Q17. A PersistentVolume has `persistentVolumeReclaimPolicy: Retain`. What happens when the bound PersistentVolumeClaim is deleted?

- [ ] A. The PV is immediately deleted and underlying cloud storage destroyed.
- [ ] B. The PV enters `Released` status; storage and data remain intact on disk.
- [ ] C. The PV enters `Available` status and binds to any new PVC.
- [ ] D. The PV is formatted with a fresh filesystem.

DOMAIN 5: TROUBLESHOOTING (30%)

Q18. A worker node shows status `NotReady`. You SSH into the node and run `systemctl status kubelet`. The service is failed. What is the most effective command to view the crash stack trace?

- [ ] A. `kubectl logs kubelet`
- [ ] B. `journalctl -u kubelet -e --no-pager`
- [ ] C. `cat /var/log/syslog | grep error`
- [ ] D. `crictl logs kubelet`

Q19. `kubectl` commands return `The connection to the server <ip>:6443 was refused`. You SSH into the control plane host. How do you view logs of the crashed API server?

- [ ] A. `kubectl logs kube-apiserver -n kube-system`
- [ ] B. `systemctl status kube-apiserver`
- [ ] C. `crictl ps -a --name kube-apiserver` followed by `crictl logs <id>`
- [ ] D. `tail -f /var/log/messages`

Q20. You are executing an etcd snapshot restore on the control plane node. Where should the restored data directory be placed?

- [ ] A. Directly over `/var/lib/etcd` without flags.
- [ ] B. A brand new directory (e.g. `/var/lib/etcd-restored`) specified by `--data-dir`.
- [ ] C. Inside `/tmp/etcd/`
- [ ] D. Over `/etc/kubernetes/pki/`

Q21. After restoring etcd to `/var/lib/etcd-restored`, what must be updated so kubelet launches the restored database?

- [ ] A. `kubectl edit pod etcd -n kube-system`
- [ ] B. Modify the `hostPath.path` in `/etc/kubernetes/manifests/etcd.yaml`.
- [ ] C. `kubeadm init phase etcd`
- [ ] D. Restart containerd.

Q22. Workload Pods cannot resolve DNS names. CoreDNS pods are in `CrashLoopBackOff` logging `Loop (127.0.0.1:53 -> :53) detected`. What is the root cause?

- [ ] A. CoreDNS image is corrupted.
- [ ] B. Host `/etc/resolv.conf` nameserver points to local loopback (127.0.0.53) causing an infinite forwarding loop.
- [ ] C. Kubelet is missing the `--cluster-dns` flag.
- [ ] D. CNI plugin is not installed.

Q23. A Pod is stuck in `Pending`. `kubectl describe pod` shows `0/3 nodes are available: 3 node(s) had untolerated taint {dedicated: gpu}`. What action resolves this?

- [ ] A. Add a matching toleration for `dedicated: gpu` in the Pod's `spec.tolerations`.
- [ ] B. Increase the Pod's CPU request.
- [ ] C. Change `restartPolicy` to `Never`.
- [ ] D. Re-create the Pod in the `kube-system` namespace.

Q24. You run `kubectl get pods` and notice a Pod in `CreateContainerConfigError`. What is the immediate diagnostic step?

- [ ] A. `kubectl logs <pod-name>`
- [ ] B. `kubectl describe pod <pod-name>` to identify missing ConfigMap or Secret keys.
- [ ] C. Restart kubelet.
- [ ] D. Drain the worker node.

Q25. A Service of type `ClusterIP` has 0 endpoints (`ENDPOINTS: <none>`). Pods are running. What is the most common cause?

- [ ] A. kube-proxy is stopped.
- [ ] B. Service `spec.selector` labels do not match the Pod `metadata.labels`, or Pods are failing readiness probes.
- [ ] C. Node firewall is blocking port 443.
- [ ] D. The Service lacks a cluster-admin RoleBinding.

3. ANSWER KEY & SCORING MATRIX

Question	Correct Answer	Domain	Points
Q1	**B** (`--upload-certs`)	Cluster Architecture	4
Q2	**B** (2 members)	Cluster Architecture	4
Q3	**B** (`kubectl drain ...`)	Cluster Architecture	4
Q4	**C** (Role in `staging` , RoleBinding in `staging`)	Cluster Architecture	4
Q5	**B** (`kubeadm certs renew all`)	Cluster Architecture	4
Q6	**A** (`kubectl rollout undo ... --to-revision=1`)	Workloads & Scheduling	4
Q7	**B** (OOMKilled / memory limit exceeded)	Workloads & Scheduling	4
Q8	**B** (`podAntiAffinity` with `hostname` topology)	Workloads & Scheduling	4
Q9	**B** (2 pods run concurrently)	Workloads & Scheduling	4
Q10	**B** (`auth-svc.security.svc.cluster.local`)	Services & Networking	4
Q11	**C** (`Gateway`)	Services & Networking	4
Q12	**B** (`ReferenceGrant`)	Services & Networking	4
Q13	**B** (Default Deny Ingress)	Services & Networking	4
Q14	**B** (Single item is AND; separate hyphens is OR)	Services & Networking	4
Q15	**B** (When Pod is scheduled to a node)	Storage	4
Q16	**D** (`ReadWriteOncePod`)	Storage	4
Q17	**B** (Enters `Released` ; data remains intact)	Storage	4
Q18	**B** (`journalctl -u kubelet -e --no-pager`)	Troubleshooting	4
Q19	**C** (`crictl ps -a` and `crictl logs`)	Troubleshooting	4
Q20	**B** (New directory specified by `--data-dir`)	Troubleshooting	4
Q21	**B** (Update `hostPath.path` in `etcd.yaml`)	Troubleshooting	4
Q22	**B** (Host loopback DNS loop)	Troubleshooting	4
Q23	**A** (Add matching toleration to Pod)	Troubleshooting	4
Q24	**B** (`kubectl describe pod` for missing CM/Secret)	Troubleshooting	4
Q25	**B** (Selector mismatch or unready probes)	Troubleshooting	4

4. SCORE INTERPRETATION & PERSONALIZED ROADMAP

Score: 88–100 Points (Advanced / Exam Ready)

- **Status:** You possess strong theoretical and administrative fundamentals.
- **Recommended Track: 30-Day Accelerated Sprint** (Focus on Volume II speed drills, Volume IV JSONPath recipes, and Volume V Mock Exams 3 & 4).

Score: 68–84 Points (Intermediate / Competent)

- **Status:** You understand Kubernetes architecture but have blind spots in advanced networking (Gateway API, ReferenceGrant) or Linux-level static pod/etcd troubleshooting.
- **Recommended Track: 45-Day Standard Track** (Deep dive into Volume I Parts II, IV, and VI; complete all 67 labs in Volume II).

Score: < 68 Points (Beginner / Foundational)

- **Status:** Needs structured foundational study across Linux kernel namespaces, declarative YAML, and core controllers.
- **Recommended Track: 60-Day Comprehensive Track** (Start with Volume I Chapters 1–8; complete Volume II beginner labs; follow the progressive schedule).

30-DAY AND 60-DAY STRUCTURED CKA STUDY PLANS

SYSTEMATIC PREPARATION BLUEPRINTS BY LEARNER PROFILE

1. TRACK OVERVIEW & METHODOLOGY

Passing the CKA on the first attempt requires consistent daily hands-on muscle memory rather than passive reading. Select the track that matches your baseline assessment score:

- **Track A: 60-Day Comprehensive Track** (Recommended for beginners and sysadmins new to containers).
- **Track B: 30-Day Accelerated Track** (Recommended for working DevOps and cloud engineers with Docker/Kubernetes exposure).

2. TRACK A: THE 60-DAY COMPREHENSIVE BLUEPRINT

Time Commitment: 1.5 to 2 hours per day (Total: ~100 hours).

```
┌──────────────────────────────────────────────────────────────────────┐
│ WEEKS 1–2: Linux Foundations, Containers & Architecture (Vols I & III) │
├──────────────────────────────────────────────────────────────────────┤
│ Day 01: Vol I Ch 01–02: Linux Namespaces, Cgroups, Containerd & CRI  │
│ Day 02: Vol I Ch 03–04: Control Plane vs. Worker Nodes & Static Pods │
│ Day 03: Vol I Ch 05–06: API Groups, OpenAPI Schema & Declarative YAML │
│ Day 04: Vol I Ch 07: Imperative kubectl Generators & Speed Aliases   │
│ Day 05: Vol I Ch 08: Linux Networking, veth Pairs, CNI Bridges       │
│ Day 06: Vol II Labs 01–05: Shell productivity and basic pod generation │
│ Day 07: Weekly Review: Flashcard Deck 1 (Core Architecture & kubectl) │
│ Day 08: Vol I Ch 09–11: Kubeadm Architecture & containerd SystemdCgroup │
│ Day 09: Vol I Ch 12–14: Initializing Clusters & Joining Worker Nodes │
│ Day 10: Vol II Labs 01–05 (Execution in Kind / Local VM)             │
│ Day 11: Vol I Ch 15: RBAC Roles, ClusterRoles, RoleBindings          │
│ Day 12: Vol I Ch 16: ServiceAccounts & Projected Tokens              │
│ Day 13: Vol II Labs 01–03: Multi-Namespace RBAC Mastery              │
│ Day 14: Weekly Review: Mock Exam 1 Diagnostic Run (Untimed)          │
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────┐
│ WEEKS 3–4: Workloads, Scheduling & Storage (Vols I & II)             │
├──────────────────────────────────────────────────────────────────────┤
│ Day 15: Vol I Ch 24–25: Pod Lifecycle, Init Containers & Sidecars    │
│ Day 16: Vol I Ch 26: Deployments, ReplicaSets & Rolling Update Math  │
│ Day 17: Vol I Ch 27–28: DaemonSets, StatefulSets, Jobs & CronJobs    │
│ Day 18: Vol I Ch 29: ConfigMaps, Secrets, Projected Volumes          │
│ Day 19: Vol II Labs 16–23: Workload Deployment Drills                │
│ Day 20: Vol I Ch 30: CPU/Mem Requests, Limits & CFS Quotas           │
│ Day 21: Weekly Review: Flashcard Deck 2 (Workloads & Scheduling)     │
│ Day 22: Vol I Ch 31: Liveness, Readiness & Startup Probes            │
│ Day 23: Vol I Ch 32–35: NodeAffinity, Taints, Tolerations, PodAntiAffinity │
│ Day 24: Vol I Ch 36–38: PriorityClasses, Preemption, HPA & ResourceQuotas │
│ Day 25: Vol II Labs 24–30: Advanced Scheduling Scenarios             │
│ Day 26: Vol I Ch 50–54: PV, PVC, StorageClasses & Access Modes       │
│ Day 27: Vol I Ch 55–59: Dynamic Provisioning, CSI & ReadWriteOncePod │
│ Day 28: Vol II Labs 46–52: Complete Storage Domain Workbook          │
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────┐
│ WEEKS 5–6: Services, Networking, Ingress & Gateway API               │
├──────────────────────────────────────────────────────────────────────┤
│ Day 29: Vol I Ch 39–41: Pod-to-Pod Packet Flows, VXLAN & CNI Architecture │
│ Day 30: Vol I Ch 42–43: ClusterIP, NodePort, LoadBalancer & kube-proxy │
│ Day 31: Vol I Ch 44–45: CoreDNS Resolution & EndpointSlices          │
│ Day 32: Vol II Labs 31–35: Core Services and DNS Diagnostics         │
│ Day 33: Vol I Ch 46: Ingress Controllers & Ingress Rules             │
│ Day 34: Vol I Ch 47: Gateway API (GatewayClass, Gateway, HTTPRoute)  │
│ Day 35: Weekly Review: Flashcard Deck 3 (Services & Networking)      │
│ Day 36: Vol I Ch 48: Fine-Grained NetworkPolicies (Default Deny, Whitelist) │
│ Day 37: Vol II Labs 36–40: Ingress & Gateway API Hands-on            │
│ Day 38: Vol II Labs 41–45: NetworkPolicy Engineering Drills          │
│ Day 39: Vol I Ch 21: etcd Raft Quorum & 7-Step Snapshot Restore      │
│ Day 40: Vol II Labs 06–07: etcd Backup and Disaster Recovery Restore │
│ Day 41: Vol I Ch 17–18: Cluster Upgrades & Safe Node Drains          │
│ Day 42: Vol II Labs 04–05: Kubeadm Control Plane and Worker Node Upgrades │
└──────────────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────────────┐
```

```
| WEEKS 7–8: Troubleshooting Mastery, Mock Exams & Exam Simulation          |
|---------------------------------------------------------------------------|
| Day 43: Vol I Ch 60–63: Hierarchical Triage, Pod & Node Failure Recovery  |
| Day 44: Vol I Ch 64–66: Kubelet Systemd & containerd Socket Troubleshooting |
| Day 45: Vol I Ch 67–68: Control Plane Crash & etcd Outage Resolution      |
| Day 46: Vol I Ch 69–72: DNS Loops, Broken NetworkPolicies, Chaos Drills   |
| Day 47: Vol II Labs 53–60: Infrastructure Troubleshooting Labs            |
| Day 48: Vol II Labs 61–67: Advanced Multi-Tier Failure Recovery           |
| Day 49: Weekly Review: Vol III Visual Atlas Troubleshooting Flowcharts    |
| Day 50: Vol V Mock Exam 1: Timed Simulation (Goal: >80% in 100 min)       |
| Day 51: Review Mock Exam 1 Scorecard & Log Weaknesses in Mistake Journal  |
| Day 52: Vol V Mock Exam 2: Timed Simulation (Goal: >80% in 100 min)       |
| Day 53: Review Mock Exam 2 Scorecard & Retest Failed Tasks                |
| Day 54: Vol V Mock Exam 3: Advanced Troubleshooting Simulation            |
| Day 55: Review Mock Exam 3 Scorecard & Practice Static Pod Recovery       |
| Day 56: Vol V Mock Exam 4: Full Proctored Certification Simulation        |
| Day 57: Final Speed Drills (Volume I Chapter 79)                          |
| Day 58: Review Volume IV Field Guide & Master Cheatsheets                 |
| Day 59: Rest Day & Environment Pre-Check                                  |
| Day 60: OFFICIAL CKA EXAM DAY                                             |
└───────────────────────────────────────────────────────────────────────────┘
```

---

3. TRACK B: THE 30-DAY ACCELERATED SPRINT

Time Commitment: 2.5 to 3 hours per day (Total: ~80 hours). Recommended for active Kubernetes operators.

- **Week 1 (Days 1–7): Core Architecture & Workload Velocity**
  - Focus: Master Volume IV imperative generators, RBAC (`kubectl auth can-i`), rolling updates, and DaemonSet/StatefulSet configurations.
  - Deliverable: Complete Volume II Labs 01–25.
- **Week 2 (Days 8–14): Networking, Gateway API, Storage & Upgrades**
  - Focus: NetworkPolicies (default-deny, cross-namespace), Gateway API HTTPRoute multi-service canary splitting, `ReadWriteOncePod`, dynamic storage, kubeadm upgrades.
  - Deliverable: Complete Volume II Labs 26–52.
- **Week 3 (Days 15–21): 30% Troubleshooting Domain Deep Dive**
  - Focus: etcd snapshot backup/restore, crashed kube-apiserver static pods, broken kubelet cgroup drivers, CoreDNS loop detection.
  - Deliverable: Complete Volume II Labs 53–67.
- **Week 4 (Days 22–30): Proctored Mock Exam Marathon**
  - Day 22: Mock Exam 1 (Timed).
  - Day 24: Mock Exam 2 (Timed).
  - Day 26: Mock Exam 3 (Advanced Troubleshooting).
  - Day 28: Mock Exam 4 (Full Certification Simulation).
  - Day 29: Final Speed Drills & PSI UI review.
  - Day 30: CKA Examination.

COMMAND MEMORY SYSTEM & SPACED-REPETITION FLASHCARDS

INSTANT CLI MUSCLE MEMORY FOR CKA EXAM SPEED

1. SPACED-REPETITION METHODOLOGY

Print or review these 24 high-yield flashcards daily. Cover the "Answer / Command" column, read the "Scenario / Challenge", and recite or type the exact syntax without looking.

2. FLASHCARD DECK: IMPERATIVE COMMAND GENERATION

Card #	Scenario / Administrative Need	Exact Command Syntax	Critical Flags & Notes
FC-01	Generate Pod YAML without creating	`kubectl run web --image=nginx:alpine $do > pod.yaml`	`$do` is `--dry-run=client -o yaml`
FC-02	Create Deployment with 3 replicas on port 80	`kubectl create deployment api --image=nginx:alpine --replicas=3 --port=80`	Creates apps/v1 Deployment
FC-03	Expose Deployment internally as ClusterIP	`kubectl expose deployment api --port=80 --target-port=8080 --name=api-svc`	Inherits deployment label selector
FC-04	Expose Deployment on NodePort 30080	`kubectl expose deployment api --type=NodePort --port=80 --target-port=8080`	Then edit service to set `nodePort: 30080`
FC-05	Update image and record revision	`kubectl set image deployment/api nginx=nginx:1.27` `kubectl annotate deployment/api kubernetes.io/change-cause="v1.27" --overwrite`	Replaces removed `--record` flag!
FC-06	Rollback deployment to specific revision	`kubectl rollout undo deployment/api --to-revision=2`	View history: `kubectl rollout history ...`
FC-07	Generate Job that runs batch command	`kubectl create job calc --image=busybox $do -- expr 10 + 20 > job.yaml`	Must specify `--` before container command
FC-08	Generate CronJob running hourly	`kubectl create cronjob backup --image=busybox --schedule="0 * * * *" $do -- date`	Uses 5-field cron syntax
FC-09	Create Secret with literal password	`kubectl create secret generic db-pass --from-literal=password='S3cur3!'`	Value is Base64 encoded automatically
FC-10	Create TLS Secret with cert and key files	`kubectl create secret tls web-tls --cert=/path/tls.crt --key=/path/tls.key`	Creates `kubernetes.io/tls` type secret
FC-11	Test permissions without switching user	`kubectl auth can-i create pods -n prod --as=system:serviceaccount:ci:builder`	Returns `yes` or `no`
FC-12	Safely evict pods prior to node maintenance	`kubectl drain node-01 --ignore-daemonsets --delete-emptydir-data --force`	Cordon automatically applied by drain

3. FLASHCARD DECK: CLUSTER ADMINISTRATION & ETCD

Card #	Scenario / Administrative Need	Exact Command Syntax	Critical Flags & Notes
FC-13	Save etcd snapshot to file with mTLS	`ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt --key=/etc/kubernetes/pki/etcd/server.key snapshot save /opt/snap.db`	All 3 TLS cert flags are mandatory
FC-14	Verify etcd snapshot integrity	`ETCDCTL_API=3 etcdctl --write-out=table snapshot status /opt/snap.db`	Checks hash, revision, and size
FC-15	Restore etcd snapshot to new directory	`ETCDCTL_API=3 etcdctl snapshot restore /opt/snap.db --data-dir=/var/lib/etcd-new --initial-cluster="m=https://127.0.0.1:2380" --initial-cluster-token="t" --name="m"`	Never restore directly over `/var/lib/etcd`
FC-16	Check kubeadm certificate expiration	`sudo kubeadm certs check-expiration`	Reports residual time for all PKI certs
FC-17	Renew all control plane certificates	`sudo kubeadm certs renew all`	Remember to copy `admin.conf` to `~/.kube/config`!
FC-18	Upgrade kubeadm control plane	`sudo kubeadm upgrade plan` `sudo kubeadm upgrade apply v1.31.2 -y`	Control plane must be upgraded before workers

4. FLASHCARD DECK: JSONPATH, TROUBLESHOOTING & LINUX

Card #	Scenario / Administrative Need	Exact Command Syntax	Critical Flags & Notes
FC-19	Sort all pods by creation timestamp	`kubectl get pods -A --sort-by=.metadata.creationTimestamp`	Instant view of newly restarted pods
FC-20	Extract Pod IPs in a single tab-separated line	`kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.podIP}{"\n"}{end}'`	JSONPath range loop syntax
FC-21	List static containers running on host	`sudo crictl ps -a --name kube-apiserver`	Primary tool when apiserver is down
FC-22	View kubelet systemd journal crash log	`sudo journalctl -u kubelet -n 50 --no-pager`	Bypasses pager for instant inspection
FC-23	Test TCP port connectivity with 3s timeout	`nc -zvw3 192.168.1.10 6443`	Tests if port is open and listening
FC-24	Instant temporary debug pod for DNS	`kubectl run net-test --rm -it --image=busybox:1.28 -- nslookup kubernetes.default`	`--rm` automatically cleans up pod

TROUBLESHOOTING DECISION TREES CATALOG

THE 6 COMPREHENSIVE INCIDENT RESPONSE PROCEDURES

1. INCIDENT TREE 1: NODE NOTREADY RECOVERY

```
Symptom: Node status reports NotReady in 'kubectl get nodes'
  |
  ├── 1. Initial Check: kubectl describe node <node> | grep -A 5 Conditions
  |     ├── DiskPressure = True ──> Clean disk: sudo crictl rmi --prune
  |     ├── MemoryPressure = True ──> Identify OOM leak, kill rogue containers
  |     └── Ready = False / Unknown ──> Node agent (kubelet) communication severed
  |
  ├── 2. SSH to Node & Inspect Service:
  |     sudo systemctl status kubelet
  |     ├── Active: inactive (dead) ──> sudo systemctl restart kubelet
  |     └── Active: failed (Result: exit-code) ──> sudo journalctl -u kubelet -e --no-pager
  |
  ├── 3. Analyze Kubelet Failure Reason:
  |     ├── "swap is enabled" ──> sudo swapoff -a && sed -i '/swap/d' /etc/fstab
  |     ├── "cgroup driver mismatch" ──> Set SystemdCgroup = true in /etc/containerd/config.toml
  |     ├── "connection refused to 6443" ──> Control plane unreachable; check firewall/routing
  |     └── "certificate has expired" ──> Renew /var/lib/kubelet/pki/ certs via bootstrap
  |
  └── 4. Verification:
        sudo systemctl restart containerd && sudo systemctl restart kubelet
        kubectl get nodes <node> (Transitions to Ready within 30 seconds)
```

2. INCIDENT TREE 2: CONTROL PLANE APISERVER DOWN (PORT 6443 REFUSED)

```
Symptom: kubectl returns "The connection to server <ip>:6443 was refused"
  |
  ├── 1. SSH to Control Plane Host:
  |     sudo systemctl is-active kubelet
  |     └── If inactive ──> sudo systemctl restart kubelet
  |
  ├── 2. Inspect Static Pod Containers via Runtime:
  |     sudo crictl ps -a --name kube-apiserver
  |     ├── Container is Exited / Crashing ──> sudo crictl logs <container-id>
  |     └── Container is Missing ──> Verify /etc/kubernetes/manifests/kube-apiserver.yaml
  |
  ├── 3. Identify Root Error from Logs:
  |     ├── "etcdserver: request timed out" ──> etcd is dead on port 2379; check etcd container
  |     ├── "unknown flag: --xxx" ──> Typo in static pod manifest args; fix YAML syntax
  |     └── "tls: bad certificate" ──> Expired PKI cert; run: sudo kubeadm certs renew all
  |
  └── 4. Verification:
        curl -k https://127.0.0.1:6443/healthz (Returns "ok")
        kubectl get nodes (Restores full CLI operation)
```

3. INCIDENT TREE 3: POD CRASHLOOPBACKOFF & EXIT CODE ANALYSIS

```
Symptom: Pod continuously crashes; restartCount increases rapidly
  |
  ├── 1. Fetch Previous Crash Log:
  |     kubectl logs <pod-name> -n <ns> --previous
  |     └── If empty ──> kubectl logs <pod-name> -n <ns>
  |
  ├── 2. Inspect Last State & Termination Reason:
  |     kubectl describe pod <pod-name> -n <ns>
  |     ├── Exit Code 0 ──> Container finished work; add long-running daemon / sleep
  |     ├── Exit Code 1 ──> Application exception, syntax error, or unhandled promise
  |     ├── Exit Code 137 ──> OOMKilled; increase resources.limits.memory in pod spec
  |     ├── Exit Code 139 ──> Segmentation fault in compiled binary
  |     ├── Exit Code 143 ──> Gracefully terminated by external SIGTERM
  |     └── Exit Code 255 ──> Entrypoint / CMD executable not found in image
  |
  └── 3. Verification:
        kubectl get pod <pod-name> -n <ns> (Status remains Running for >60 seconds)
```

4. INCIDENT TREE 4: SERVICE CONNECTIVITY & MISSING ENDPOINTS

```
Symptom: curl http://<service-ip>:<port> drops packets or times out
  |
  ├── 1. Inspect Endpoints:
  |     kubectl get endpoints <service-name> -n <ns>
  |     |
  |     ├── If ENDPOINTS: <none>
  |     |   ├── Check Labels: Compare service spec.selector with pod metadata.labels
  |     |   └── Check Readiness: Are pods passing readiness probes? (Unready = Stripped)
  |     |
  |     └── If ENDPOINTS: Populated with Pod IPs
  |         ├── Direct Connect: Test curl http://<pod-ip>:<targetPort> from another pod
  |         |   ├── If Direct Pod Fails ──> Container app not listening on targetPort
  |         |   └── If Direct Pod Works ──> kube-proxy iptables rule programming failed
  |         |
  |         └── Check NetworkPolicies:
  |             kubectl get netpol -n <ns> (Look for policies dropping ingress traffic)
  |
  └── 2. Verification:
        kubectl run curl-test --rm -it --image=curlimages/curl -- curl -Iv http://<service-name>.<ns>
```

5. INCIDENT TREE 5: COREDNS NAME RESOLUTION OUTAGE

```
Symptom: Pods cannot resolve service names (e.g. "nslookup kubernetes.default" fails)
  |
  ├── 1. Inspect CoreDNS Pods in kube-system:
  |     kubectl get pods -n kube-system -l k8s-app=kube-dns
  |     ├── Status: CrashLoopBackOff ──> Check logs: kubectl logs -n kube-system -l k8s-app=kube-dns
  |     |   └── "Loop detected" ──> Host /etc/resolv.conf points to 127.0.0.53; fix host DNS
  |     |
  |     └── Status: Running ──> Inspect CoreDNS Service
  |         kubectl get svc kube-dns -n kube-system (Verify ClusterIP: 10.96.0.10)
  |         kubectl get endpoints kube-dns -n kube-system (Verify endpoints exist)
  |
  ├── 2. Test CoreDNS Direct Query from Debug Pod:
  |     kubectl run dns-test --rm -it --image=busybox:1.28 -- nslookup 10.96.0.1 10.96.0.10
  |     ├── Succeeds on IP, fails on names ──> /etc/resolv.conf search path missing in pod
  |     └── Timeouts completely ──> NetworkPolicy in caller namespace blocks Port 53 UDP
  |
  └── 3. Verification:
        kubectl run dns-test --rm -it --image=busybox:1.28 -- nslookup kubernetes.default
```

6. INCIDENT TREE 6: PERSISTENTVOLUMECLAIM BINDING FAILURE

```
Symptom: PersistentVolumeClaim remains in Pending status indefinitely
  │
  ├── 1. Inspect PVC Events:
  │     kubectl describe pvc <pvc-name> -n <ns>
  │     │
  │     ├── "waiting for first consumer to be created"
  │     │   └── NORMAL BEHAVIOR: StorageClass uses volumeBindingMode: WaitForFirstConsumer.
  │     │       PVC will bind automatically once a Pod referencing it is scheduled!
  │     │
  │     ├── "no persistent volumes available for this claim"
  │     │   ├── Capacity Mismatch: Existing PV capacity < requested PVC storage
  │     │   ├── AccessMode Mismatch: PV accessModes do not include PVC requested mode
  │     │   └── StorageClass Mismatch: PV storageClassName != PVC storageClassName
  │     │
  │     └── "provisioning failed"
  │         └── Dynamic Provisioning Error: CSI driver down or cloud API IAM denied
  │
  └── 2. Verification:
        kubectl get pvc <pvc-name> -n <ns> (Status transitions from Pending to Bound)
```

HANDS-ON LAB PROGRESS & MASTERY TRACKER

AUDITABLE LOG FOR ALL 67 PRACTICAL WORKBOOK LABS

1. HOW TO USE THIS TRACKER

Track every lab attempt systematically. A lab is considered **Mastered** only when executed in **under 8 minutes without looking at the solution section** and passing all automated verification checks.

- **Status Codes:** `NS` (Not Started), `IP` (In Progress), `REV` (Needs Revision), `PASS` (Completed with hints), `MASTER` (Completed timed with 0 hints).

2. DOMAIN 1: CLUSTER ARCHITECTURE, INSTALLATION & CONFIGURATION (LABS 01–15)

Lab ID	Lab Title	Target Time	Attempt 1 Date	Time Taken	Hints Used	Status	Notes / Errors Encountered
LAB 01	Multi-Namespace RBAC Role & Binding	6 min				NS	
LAB 02	Cluster-Wide Privilege with ClusterRoles	6 min				NS	
LAB 03	Worker Node Maintenance (Cordon & Drain)	5 min				NS	
LAB 04	Control Plane Version Upgrade with kubeadm	8 min				NS	
LAB 05	Worker Node Upgrade with kubeadm	6 min				NS	
LAB 06	etcd Snapshot Backup with mTLS	5 min				NS	
LAB 07	etcd Disaster Recovery Snapshot Restore	7 min				NS	
LAB 08	Deploying Applications via Helm	5 min				NS	
LAB 09	Multi-Environment Overlays with Kustomize	6 min				NS	
LAB 10	Custom Resource Definition (CRD) Triage	5 min				NS	
LAB 11	ServiceAccount Projected Token Rotation	5 min				NS	
LAB 12	Certificate Expiration & kubeadm renew	6 min				NS	
LAB 13	HA Control Plane Quorum Calculation	4 min				NS	
LAB 14	Containerd Socket & Cgroup Alignment	6 min				NS	
LAB 15	Static Pod Provisioning on Worker Node	5 min				NS	

3. DOMAIN 2: WORKLOADS & SCHEDULING (LABS 16–30)

Lab ID	Lab Title	Target Time	Attempt 1 Date	Time Taken	Hints Used	Status	Notes / Errors Encountered
LAB 16	Zero-Downtime Rolling Update & Rollback	5 min				NS	
LAB 17	StatefulSet Sequential Scaling & Headless	7 min				NS	
LAB 18	DaemonSet Node Scheduling & Tolerations	5 min				NS	
LAB 19	Batch Jobs with Parallelism & Completions	5 min				NS	
LAB 20	CronJob Concurrency Policy Enforcement	5 min				NS	
LAB 21	Native Sidecar Container Pattern	6 min				NS	
LAB 22	ConfigMap Volume Mounts & Atomic Updates	5 min				NS	
LAB 23	Secret Management & Env Injection	5 min				NS	
LAB 24	Pod Quality of Service (QoS) Tuning	4 min				NS	
LAB 25	Health Checks (Startup, Readiness, Liveness)	6 min				NS	
LAB 26	Node Affinity (Required vs Preferred)	5 min				NS	
LAB 27	Taints, Tolerations & Node Eviction	5 min				NS	
LAB 28	Pod Anti-Affinity Across Topology	6 min				NS	
LAB 29	Horizontal Pod Autoscaler (HPA) Tuning	6 min				NS	
LAB 30	ResourceQuotas & LimitRanges	6 min				NS	

4. DOMAIN 3: SERVICES & NETWORKING (LABS 31–45)

Lab ID	Lab Title	Target Time	Attempt 1 Date	Time Taken	Hints Used	Status	Notes / Errors Encountered
LAB 31	ClusterIP Creation & Endpoints Check	4 min				NS	
LAB 32	NodePort Service on Static Port Range	5 min				NS	
LAB 33	Headless Service Peer DNS Resolution	5 min				NS	
LAB 34	EndpointSlice Inspection & Custom IP	5 min				NS	
LAB 35	CoreDNS Custom Forwarding ConfigMap	6 min				NS	
LAB 36	Ingress Path-Based Routing Rules	6 min				NS	
LAB 37	Ingress TLS Secret Termination	5 min				NS	
LAB 38	Gateway API GatewayClass & Gateway Setup	6 min				NS	
LAB 39	HTTPRoute Weighted Traffic Canary Split	6 min				NS	
LAB 40	HTTPRoute Cross-Namespace ReferenceGrant	7 min				NS	
LAB 41	Default-Deny All Ingress NetworkPolicy	4 min				NS	
LAB 42	Pod & Namespace Selector Whitelisting	6 min				NS	
LAB 43	Egress NetworkPolicy with DNS Allow	6 min				NS	
LAB 44	External CIDR Ingress Filtering (ipBlock)	5 min				NS	
LAB 45	Live Packet Tracing with Ephemeral Pods	5 min				NS	

5. DOMAIN 4: STORAGE (LABS 46–52)

Lab ID	Lab Title	Target Time	Attempt 1 Date	Time Taken	Hints Used	Status	Notes / Errors Encountered
LAB 46	Static PV and PVC 1-to-1 Binding	5 min				NS	
LAB 47	Dynamic StorageClass Provisioning	5 min				NS	
LAB 48	Delayed Volume Binding (WaitForFirstConsumer)	5 min				NS	
LAB 49	Reclaim Policy (Retain vs Delete)	5 min				NS	
LAB 50	Single-Pod Data Lock with ReadWriteOncePod	5 min				NS	
LAB 51	Dynamic PVC Online Volume Expansion	5 min				NS	
LAB 52	Multi-Pod Shared Storage (ReadWriteMany)	6 min				NS	

6. DOMAIN 5: TROUBLESHOOTING (LABS 53–67)

Lab ID	Lab Title	Target Time	Attempt 1 Date	Time Taken	Hints Used	Status	Notes / Errors Encountered
LAB 53	CSI Driver & VolumeAttachment Triage	6 min				NS	
LAB 54	Pod Stuck in Pending (Resource/Taints)	5 min				NS	
LAB 55	Diagnosing & Fixing CrashLoopBackOff	5 min				NS	
LAB 56	ImagePullBackOff & Registry Credentials	5 min				NS	
LAB 57	OOMKilled (Exit Code 137) Recovery	4 min				NS	
LAB 58	Triage & Recovery of NotReady Worker Node	6 min				NS	
LAB 59	Broken Kubelet Cgroup Driver Alignment	5 min				NS	
LAB 60	Broken Control Plane Static Pod Repair	6 min				NS	
LAB 61	Expired Cluster PKI Certificate Recovery	6 min				NS	
LAB 62	CoreDNS Loop Crash Recovery	5 min				NS	
LAB 63	Network Service Missing Endpoints Fix	5 min				NS	
LAB 64	Broken Cross-Namespace NetworkPolicy	6 min				NS	
LAB 65	etcd Quorum Loss & Database Alarm Clear	7 min				NS	
LAB 66	Integrated Multi-Failure Grand Challenge	15 min				NS	
LAB 67	Advanced Chaos Recovery: Cross-NS Gateway	12 min				NS	

CKA CANDIDATE MISTAKE JOURNAL & POST-MORTEM SYSTEM

STRUCTURED ROOT-CAUSE AUDITING TO PREVENT EXAM REPEAT ERRORS

---

1. THE MISTAKE JOURNAL PROTOCOL

Research proves that learners who document and categorize their coding mistakes retain correct procedures 3x longer than those who simply read the solution.

Whenever you fail a lab or mock exam task, complete this 5-point root-cause template:

1. **Error Event:** Exactly what command was run and what error output was returned.
2. **Category of Error:** Syntax typo, namespace oversight, context mismatch, conceptual misconception, or documentation inability.
3. **True Root Cause:** Why the error occurred mechanically inside Kubernetes.
4. **Corrective Procedure:** The exact verified command sequence that permanently fixes it.
5. **Prevention Rule:** A one-sentence rule you will recite to never make this mistake again.

---

2. COMMON HIGH-FREQUENCY MISTAKE REGISTER

Mistake 1: The Context Amnesia Error

- **Symptom:** Task asks to fix node in cluster `infra-cluster`, but candidate executes commands against `k8s`. Result: 0 points awarded on official exam.
- **Root Cause:** Failing to run the context switch command specified at the top of the question.
- **Corrective Procedure:**

```
kubectl config use-context infra-cluster
kubectl config current-context # Verify
```

- **Prevention Rule:** *Never type a single kubectl command without first copying and executing the task's context switch line.*

Mistake 2: The Deprecated Flag Trap

- **Symptom:** Running `kubectl set image deployment/app nginx=nginx:1.27 --record` produces warnings or fails to record revision history.
- **Root Cause:** `--record` was permanently removed in kubectl v1.26+.
- **Corrective Procedure:**

```
kubectl set image deployment/app nginx=nginx:1.27
kubectl annotate deployment/app kubernetes.io/change-cause="Upgrade to 1.27" --overwrite
```

- **Prevention Rule:** *Always use kubernetes.io/change-cause annotations to record deployment history.*

Mistake 3: The etcd Direct Overwrite Disaster

- **Symptom:** Executing `etcdctl snapshot restore /opt/backup.db` directly restores over `/var/lib/etcd`, causing corrupted Raft logs and an unbootable control plane.
- **Root Cause:** Omitting `--data-dir=/var/lib/etcd-restored` forces etcd to write into the active database path.
- **Corrective Procedure:**

```
ETCDCTL_API=3 etcdctl snapshot restore /opt/backup.db \
  --data-dir=/var/lib/etcd-restored \
  --initial-cluster="master=https://127.0.0.1:2380" \
  --name="master"
# Update /etc/kubernetes/manifests/etcd.yaml hostPath to /var/lib/etcd-restored
```

- **Prevention Rule:** *Always restore etcd snapshots to a brand new target directory and point the static pod manifest to that new path.*

Mistake 4: The NetworkPolicy Egress DNS Lockout

- **Symptom:** You apply an egress NetworkPolicy to isolate a pod. Suddenly, the pod throws `Name or service not known` and cannot communicate with any internal service.
- **Root Cause:** Applying any egress policy turns on default-deny for all outbound traffic, silently blocking UDP Port 53 queries to CoreDNS.
- **Corrective Procedure:** Always append an explicit DNS egress rule to every egress NetworkPolicy:

```
egress:
- ports:
  - protocol: UDP
    port: 53
  - protocol: TCP
    port: 53
```

- **Prevention Rule:** *Every egress NetworkPolicy must explicitly whitelist UDP port 53 to allow CoreDNS resolution.*

---

3. BLANK MISTAKE JOURNAL TEMPLATE FOR CANDIDATE USE

```
### MISTAKE ENTRY #[NUMBER]
* **Date & Lab/Mock Task:** [e.g., Day 12 / Vol II Lab 42]
* **Target Objective:** [e.g., Restricting ingress traffic to backend pods]
* **What I Typed / Ran:**
  ```bash
  [Insert erroneous command or manifest snippet here]
```

- **What Kubernetes Returned:**

```
[Insert error message, HTTP status code, or unexpected behavior]
```

- **Why It Failed Mechanically:** [Explain the internal reason: e.g., Missing hyphen in YAML created AND instead of OR]
- **The Correct Solution:**

```
[Insert verified working command]
```

- **My Prevention Rule:** "[State your memorable takeaway rule]"
```

```
<div class='page-break'></div>

# THE KUBERNETES ADMINISTRATOR INTERVIEW GUIDE
## BRIDGING CKA CERTIFICATION TO SENIOR DEVOPS & SRE JOB INTERVIEWS

---

## 1. CKA OBJECTIVES VS. REAL-WORLD PRODUCTION INTERVIEW TOPICS

| Topic Dimension | CKA Examination Focus | Real-World SRE / DevOps Job Interview Focus |
| :--- | :--- | :--- |
| **Cluster Setup** | Manual single-node `kubeadm init`, join commands | Terraform, GitOps (ArgoCD/Flux),
| **Configuration** | Raw YAML manifests, imperative `kubectl` generators | Helm charts, Kustomize multi-
| **Networking** | Basic CNI install, Calico NetworkPolicy, Gateway API HTTPRoute | Service mesh (Istio/L
| **Storage** | PV/PVC binding, `ReadWriteOncePod`, local hostPath | CSI snapshot lifecycles, cross-regio
| **Observability** | `kubectl top`, `journalctl -u kubelet`, `crictl logs` | Prometheus operator, Grafan
| **Security** | Basic RBAC Roles/Bindings, ServiceAccount tokens | Kyverno/OPA Gatekeeper admission poli

---

## 2. TOP 10 PRODUCTION INTERVIEW QUESTIONS & ARCHITECTURAL ANSWERS

### Q1: "Walk me through what happens when an engineer runs `kubectl apply -f deployment.yaml` until the
**Expected Senior Answer:**
1. **Client & API Layer:** `kubectl` performs client-side validation, sends an HTTPS POST/PUT to `kube-ap
2. **Authentication & Authorization:** The API server authenticates the client certificate/token, then ch
3. **Admission Controllers:** Mutating admission webhooks run (injecting defaults, sidecars), followed by
4. **etcd Write:** The Deployment object is serialized and committed to `etcd` via Raft consensus.
5. **Deployment Controller:** The Deployment Controller detects the new object via watch streams, compute
6. **ReplicaSet Controller:** Detects the ReplicaSet and creates unassigned Pod specifications (with `spe
7. **kube-scheduler:** Scheduler filter plugins evaluate node capacity, taints, and affinity. Scoring plu
8. **Kubelet SyncLoop:** The kubelet on the target node receives the pod assignment via watch stream.
9. **CRI Runtime Call:** Kubelet calls `containerd` via CRI gRPC. Containerd requests `runc` to configure
10. **CNI Network Setup:** The CNI plugin creates a `veth` pair, connects one end to the container netns,
11. **Storage Mounts:** VolumeManager calls CSI plugins to attach and mount required volumes.
12. **Container Launch & Probes:** The application container starts. Kubelet runs startup and readiness p
13. **Endpoint Injection:** The EndpointSlice controller detects the ready pod, adds its IP to the Servic

---

### Q2: "A production node suddenly transitions to `NotReady`. How do you triage it under a high-severity
**Expected Senior Answer:**
1. **Assess Impact & PDBs:** First check `kubectl get pods -o wide --field-selector spec.nodeName=<node>`
2. **Examine Conditions:** `kubectl describe node <node>`. Check `DiskPressure`, `MemoryPressure`, `PIDPr
3. **SSH Triage:**
   * Run `systemctl is-active kubelet` and `systemctl is-active containerd`.
   * Check kernel dmesg: `dmesg -T | grep -E "oom|killed|error"`.
   * Check journal logs: `journalctl -u kubelet -n 100 --no-pager`.
4. **Network Verification:** Test if the node can reach the API server: `curl -k https://<cp-ip>:6443/hea
5. **Safe Resolution:**
   * If swap was accidentally turned on after reboot: `swapoff -a && systemctl restart kubelet`.
   * If disk full: run `crictl rmi --prune` and clear dead container logs under `/var/log/pods/`.
   * If hardware failure: run `kubectl drain <node> --ignore-daemonsets --delete-emptydir-data` to resche

---
```

```
### Q3: "What are the mechanical differences between `iptables` mode and `IPVS` mode in `kube-proxy`, and
**Expected Senior Answer:**
* **iptables:** Operates as a flat, sequential list of packet filter rules. Every service and endpoint ad
* **IPVS (IP Virtual Server):** Implements transport-layer load balancing inside the Linux kernel using h
* **When to switch:** Switch to IPVS in clusters scaling beyond 1,000 services or when connection rebalan

---

### Q4: "How does the Raft consensus algorithm work in etcd, and why does an etcd cluster always require
**Expected Senior Answer:**
* etcd uses Raft to maintain a replicated, consistent state machine across distributed nodes.
* Operations require a strict quorum: $\lfloor N/2 \rfloor + 1$ active members.
* **Why odd numbers:**
  * A 3-node cluster requires 2 votes for quorum; it tolerates 1 failure ($3 - 2 = 1$).
  * A 4-node cluster requires 3 votes for quorum; it *still* only tolerates 1 failure ($4 - 3 = 1$). Addi
  * A 5-node cluster requires 3 votes for quorum; it tolerates 2 failures ($5 - 3 = 2$).
  * Therefore, odd numbers ($3, 5$) maximize fault tolerance per node cost and avoid split-brain split vo

---

### Q5: "How does Kubernetes enforce CPU limits vs. Memory limits at the Linux kernel level?"
**Expected Senior Answer:**
* **CPU Limits:** Enforced via the Linux kernel **Completely Fair Scheduler (CFS) bandwidth control** (`c
* **Memory Limits:** Enforced via **memory cgroup ceilings** (`memory.max` in cgroups v2). When a contain


<div class='page-break'></div>

# QUICK REVISION SYSTEM & THE LAST-7-DAYS SPRINT PLAN
## CONDENSED HIGH-YIELD FORMULAS & FINAL WEEK EXAM PROTOCOL

---

## 1. THE 7-DAY PRE-EXAM SPRINT PROTOCOL

| Day | Focus Area | Required Daily Actions | Target Mastery Metric |
| :---: | :--- | :--- | :--- |
| **Day -7** | Domain 1: Cluster Setup & RBAC | Re-execute Labs 01, 04, 06, 07 (kubeadm upgrade & etcd re
| **Day -6** | Domain 2: Workloads & Scheduling | Practice writing complex affinity, taints, sidecars, an
| **Day -5** | Domain 3: Services, Ingress & Gateway | Practice multi-tier NetworkPolicies and Gateway AP
| **Day -4** | Domain 4: Storage Architecture | Practice PV/PVC with `WaitForFirstConsumer` and `ReadWrit
| **Day -3** | Domain 5: Troubleshooting Marathon | Execute Labs 58, 60, 62, 65, 66 (NotReady nodes, cras
| **Day -2** | Timed Proctored Simulation | Execute Volume V Mock Exam 4 under strict 120-minute timer. |
| **Day -1** | Review Cheatsheets & Rest | Review this Quick Revision document, test PSI browser, sleep 8
| **Day 0** | **OFFICIAL EXAM DAY** | Execute Exam Strategy (Wave 1 $\to$ Wave 2 $\to$ Wave 3); PASS CKA!

---

## 2. ONE-PAGE DOMAIN CHEAT SHEETS

### DOMAIN 1: CLUSTER ARCHITECTURE & CONFIGURATION
```bash
# 1. Kubeadm Upgrade Sequence
sudo apt-get install -y kubeadm=1.31.2-1.1
sudo kubeadm upgrade plan
sudo kubeadm upgrade apply v1.31.2 -y
kubectl drain <node> --ignore-daemonsets
sudo apt-get install -y kubelet=1.31.2-1.1 kubectl=1.31.2-1.1
sudo systemctl daemon-reload && sudo systemctl restart kubelet
```
```

```
kubectl uncordon <node>

# 2. etcd Backup & Restore
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key snapshot save /opt/snap.db

ETCDCTL_API=3 etcdctl snapshot restore /opt/snap.db --data-dir=/var/lib/etcd-new \
  --initial-cluster="m=https://127.0.0.1:2380" --name="m"
# Update /etc/kubernetes/manifests/etcd.yaml hostPath to /var/lib/etcd-new
```

DOMAIN 2: WORKLOADS & SCHEDULING

```
# 1. Rolling Updates & Revision Recording
kubectl set image deployment/app nginx=nginx:1.27
kubectl annotate deployment/app kubernetes.io/change-cause="v1.27 upgrade" --overwrite
kubectl rollout undo deployment/app --to-revision=1

# 2. Key Pod Scheduling Fields
spec.nodeSelector: { disk: ssd }
spec.affinity.nodeAffinity.requiredDuringSchedulingIgnoredDuringExecution.nodeSelectorTerms...
spec.tolerations: [{ key: "dedicated", operator: "Equal", value: "gpu", effect: "NoSchedule" }]
```

DOMAIN 3: SERVICES & GATEWAY API

```
# Gateway API HTTPRoute Skeleton
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: api-route
  namespace: apps
spec:
  parentRefs:
  - name: shared-gw
    namespace: infra
  rules:
  - matches:
    - path: { type: PathPrefix, value: /api }
    backendRefs:
    - name: api-v1
      port: 8080
      weight: 90
    - name: api-v2
      port: 8080
      weight: 10
```

DOMAIN 4: STORAGE & DYNAMIC PROVISIONING

```
# WaitForFirstConsumer StorageClass & Claim
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: { name: local-sc }
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: WaitForFirstConsumer
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: db-pvc }
spec:
  storageClassName: local-sc
  accessModes: [ReadWriteOncePod]
  resources: { requests: { storage: 10Gi } }
```

DOMAIN 5: TROUBLESHOOTING RUNBOOKS

```
# First 60 Seconds Triage Commands
kubectl get nodes -o wide
systemctl status kubelet
journalctl -u kubelet -e --no-pager
sudo crictl ps -a
sudo crictl logs <id>
kubectl get pods -A -o wide --field-selector status.phase!=Running
kubectl describe pod <pod> | tail -n 20
```